FRI Natural language processing course 2026
Project report

University *of Ljubljana*
Faculty *of Computer and Information Science*

Conversational Recommender System for Vehicles

Primož Gabrovec, Rožle Sterle and Zala Urh

Abstract

The abstract goes here. The abstract goes here.

Keywords
Keyword1, Keyword2, Keyword3 ...

Advisors: Aleš Žagar

Introduction

In the past years, choosing a suitable vehicle has become very challenging due to a wide range of options. Customers can choose between many models with different fuel types, engine characteristics, size, and other specifics.

Most existing systems that assist users in selecting vehicles are based on predefined filters, such as price range, brand, transmission types, and others. While this approach is effective for experienced users, it might not be that suitable for those who cannot easily translate their needs into technical details.

As an improvement to such systems, Conversational Recommender Systems (CRS) have been introduced. A CRS is an interactive system that allows users to express their preferences through dialogue, while the system can ask follow-up questions to refine these preferences and provide the best recommendation(s).

In this paper, we propose a hybrid CRS for vehicle selection using that combines large language models (LLMs), structured constraint extraction, and retrieval-augmented generation (RAG). The system engages the user in a conversation with an LLM that helps them express and refine their vehicle preferences. The final product is a system that simplifies making a decision for non-expert users, and also improves customer experience in vehicle selling companies.

Related work

Several studies have explored recommender systems, however the vehicle domain remains limited. We categorized existing work into recommendation systems for vehicles and conversational recommender systems for various products.

In the context of vehicle recommendation, Beddouda et al. [1] proposed a system using BERT4Rec, which predicts which cars a user might be interested in based on past interactions (without any user communication). Singh et al. [2] introduced a one-turn dialogue system that uses LDA model to analyse user reviews, where users describe their car preferences and the system returns top few recommendations.

For general CRS, several multi-turn dialogue systems have been proposed, especially in the travel domain. Liao et al. [3] proposed a deep recommender system using seq2seq with a neural latent topic component. Baizal et al. [4] introduced an ontology-based system, and Carrillo et al. [5] introduced a system that uses Chatbot to collect user preferences and generates similarity-based recommendations using SBERT embeddings.

In addition to the travel domain, CRS have been implemented in several other domains. Sun et al. [6] developed a reinforcement learning based CRS for restaurant recommendation using Yelp review data. Zhu et al. [7] proposed a movie recommendation CRS using retrieval-augmented generation (RAG) approach in combination with a large language model (LLM).

Methods

The proposed system is designed as a Conversational Recommender System (CRS) that integrates large language models (LLMs), structured data processing, and retrieval-augmented generation (RAG) to provide personalized vehicle recommendations through natural language interaction. The overall pipeline is described below as well as shown on the figure 1.

Data Preprocessing

The first dataset was extracted from the CarAPI website [8]. It provides vehicle data in JSON format, such as price, fuel consumption, engine specifications... Data was then processed and stored into structured SQLite database (on Figure 1 marked as Structured database).

The second dataset consists of vehicle brochures in PDF format extracted from the Auto-Brochures website [9]. The dataset includes multiple car brands available on the US market, of which 26 were selected for this work. We used a Qwen embedding model to generate vector represenations of the dataset content. These embeddings were then indexed using FAISS (Facebook AI similarity search) [10] to enable efficient similarity search (Dataset is marked on Figure 1 as FAISS Vector Database).

Conversational Recommender System - CRS

The system begins with user describing in natural language their preferences and requirements they look for in a new car. That input is then processed by the LLM (on figure 1 marked as LLM Parser). The LLM extracts structured representations of user's preferences in a JSON-like format. An example of a parsed query is shown in a Table 1.

Name	Value	Constraint (min, max, equal...)
body_type	SUV	equal
engine_type	electric	equal
combined_l_per_100km	None	None

Table 1. Example of parsed text for the following user's query: "I'm looking for a spacious SUV with low consumption, preferably electric."

However, a single user query is often insufficient to produce high-quality recommendations. As seen in Table 1, the LLM is able to identify that a preference is important to the user, but cannot determine the exact value of the corresponding constraint. For example, a preference such as low fuel consumption may be correctly extracted but with missing a precise numerical value, which then results as an incomplete structured information.

Another problem with a single query is possibly a very large choice of cars for give user's preferences. If user wasn't able to express all their preferences or forgot to list them, that set can become really big.

To address these issues, we improved the system with creating a conversational LLM. (marked on Figure 1 with the same name). This model interacts with the user to complete missing constraints and suggest additional attributes that could be in interest to the user.

Then the system employs a dual data strategy using databases described in the previous section. First, we use parsed query in structured database using constraint-based filtering, which initially reduces the candidate set. The parsed query is then embedded and used for semantic search in the FAISS vector database. This search returns columns relevant to user's query.
TODO: dokončej to ko bo retrival.py done

After that, the system combines both retrieved sets of car candidates. Then cars from this merged set are ranked according to how well they match the user's preferences and requirements. The highest ranked cars are sent to the LLM, which generates the final recommendation response and comments on the key differences between the suggested cars.

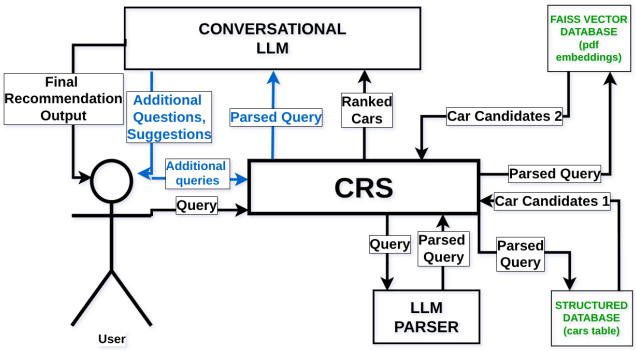

Figure 1. CRS Architecture. Blue arrows represent conversation interaction loop between the user and the LLM, green components represent databases that the system uses.

Evaluation of a model

Evaluation ideas:

Test if RAG is working well

Build a small controlled dataset (10 very different cars), where only one is meeting the requirements. Ask the system to recommend a car and check if it picks only the valid one.

Compare CRS to other LLMs

Another idea is also creating a few prompts, for which we find a set of suitable cars. Then we compare the output of our CRS system and one another LLM (preferably one that doesn't have access to web). Then we evaluate if the LLMs found any car from the set.

Compare CRS to some website

Compare recommendation to some website (Carwow, Autotrader according to chatgpt) and compare for the same requirements the models output, though it is subjective method of evaluation (hard to tell which model is better, no hard truth on this topic)

Advanced method - baseline filter model

Ideally (but too complicated to implement in such short time) would be to create a basic filtering system, and conmpare the outputs with what the LLM is offering. Again the idea wouldn't be to show it is better output objectively but either it shows more option, or wilder choices, etc. Especially if the requirements are not precise or not coherent between themselves (for instance big car + low consumption + high acceleration)

Equations
Lists
Figures
Tables
Code examples

Results

Discussion

Use the Discussion section to objectively evaluate your work, do not just put praise on everything you did, be critical and exposes flaws and weaknesses of your solution. You can also explain what you would do differently if you would be able to start again and what upgrades could be done on the project in the future.

Acknowledgments

Here you can thank other persons (advisors, colleagues ...) that contributed to the successful completion of your project.

References

[1] Amira Riham Beddouda and Bochra Belhemissi. *A VEHICLE RECOMMENDATION SYSTEM FOR SALES*. PhD thesis, KASDI MERBAH UNIVERSITY OUARGLA.

[2] Saumya Singh, Soumyadeepta Das, Ananya Sajwan, Ishanika Singh, and Ashish Alok. Car recommendation system using customer reviews. *International Research Journal of Engineering and Technology (IRJET)*, 9(10):983–989, 2022.

[3] Lizi Liao, Ryuichi Takanobu, Yunshan Ma, Xun Yang, Minlie Huang, and Tat-Seng Chua. Deep conversational recommender in travel, 2019.

[4] ZK Abdurahman Baizal, Dede Tarwidi, B Wijaya, et al. Tourism destination recommendation using ontology-based conversational recommender system. *International Journal of Computing and Digital Systems*, 10, 2021.

[5] Juan Carlos Carrillo, Victoria Beltran, Laura Sebastia, and Eva Onaindia. Smarttur+ eco: a conversational recommender system for tourism. 2023.

[6] Yueming Sun and Yi Zhang. Conversational recommender system. In *The 41st international acm sigir conference on research & development in information retrieval*, pages 235–244, 2018.

[7] Yaochen Zhu, Chao Wan, Harald Steck, Dawen Liang, Yesu Feng, Nathan Kallus, and Jundong Li. Collaborative retrieval for large language model-based conversational recommender systems. In *Proceedings of the ACM on Web Conference 2025*, pages 3323–3334, 2025.

[8] Carapi, 2026.

[9] Auto-brochures.

[10] Welcome to faiss documentation, 2026.